CITS5017 Deep Learning
Topic 9: Reinforcement Learning

Associate Professor Du Huynh
(Unit Coordinator and Lecturer)

Semester 2, 2026

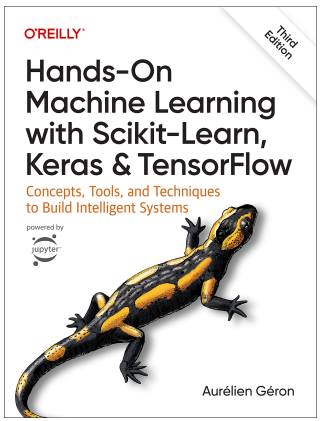


THE UNIVERSITY OF
WESTERN
AUSTRALIA

Today

Chapters 18.

Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow



Reinforcement Learning

Here are the main topics we will cover:

- Introduction to reinforcement learning and its applications
- *Rewards* in RL
- Policy search and policy gradients
- The OpenAI Gym toolkit and training a NN to create policy for the CartPole game
- Evaluating Actions (The Credit Assignment Problem) and discounted rewards
- Markov Decision Processes
- ϵ greedy policy
- Q-learning, approximate Q-learning, and deep Q-learning

Reinforcement Learning – A Brief Introduction

- Reinforcement Learning (RL) is one of the most exciting fields of Machine Learning today, and also one of the oldest.
- It has been around since the 1950s, producing many interesting applications over the years, in particular in games (e.g., TD-Gammon) and in machine control.
- In 2013, researchers from an English startup called DeepMind demonstrated that a system could learn to play just about any Atari game from scratch and eventually outperform humans in most of them, using only raw pixels as inputs and without any prior knowledge of the rules of the games.
- These amazing feats culminated in May 2017 with the victory of their system AlphaGo against Ke Jie, the world champion of the game of *Go*.
- Today the whole field of RL is boiling with new ideas, with a wide range of applications. DeepMind was bought by Google for over 500 million dollars in 2014.

How did RL algorithms do that?

With hindsight it seems rather simple: they applied the power of Deep Learning to the field of Reinforcement Learning, and it worked beyond their wildest dreams.

In this lecture we will learn the basic concepts behind Reinforcement Learning, including:

1. *policy gradients*,
2. *Markov decision processes* (MDPs), and
3. *Deep Q-network*.

The same techniques can be used for a wide variety of tasks, from walking robots to self-driving cars.

We will use these techniques to train models to balance a pole on a moving cart.

Learning to Optimize Rewards

- In Reinforcement Learning, a software *agent* makes *observations* and takes *actions* within an *environment*, and in return it receives *rewards*.
- Its objective is to learn to act in a way that will maximize its expected long-term rewards.
  We can think of positive reward as pleasure and negative reward as pain. So the agent acts in the environment and learns by trial and error to maximize its pleasure and minimize its pain.

Learning to Optimize Rewards (cont.)

A few examples

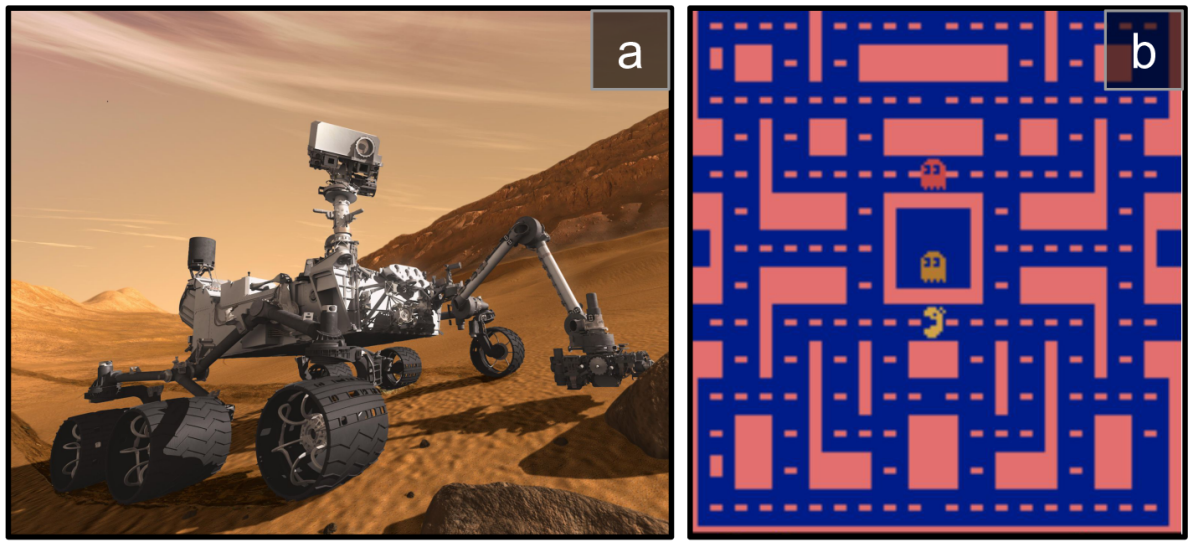


Figure 18-1. Reinforcement Learning examples: (a) robotics, (b) Ms. Pac-Man, (c) Go player, (d) thermostat, (e) automatic trader

(a) The agent can be the program controlling a walking robot, observing the real world environment through sensor readings. Its positive reward could be approaching the target destination and negative reward could be wasting time in the wrong direction or falling down.

Learning to Optimize Rewards (cont.)

A few examples (cont.)

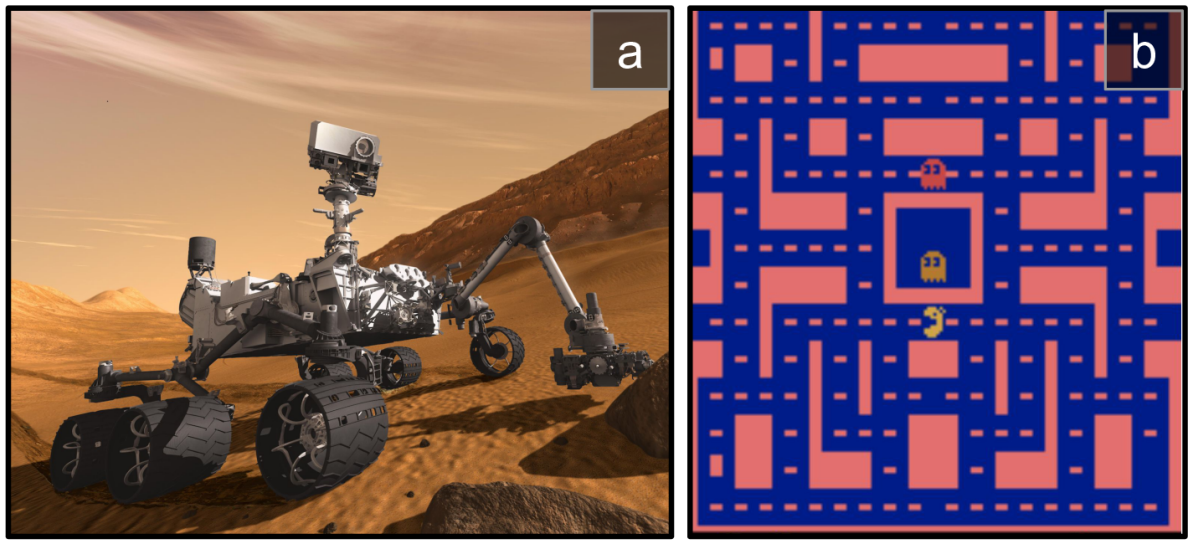


Figure 18-1. Reinforcement Learning examples: (a) robotics, (b) Ms. Pac-Man, (c) Go player, (d) thermostat, (e) automatic trader

(b) The agent can be the program controlling Ms. Pac-Man. The environment is a simulation of the Atari game; the actions are the possible joystick positions; observations are the screenshots; the rewards are just the game points.

Learning to Optimize Rewards (cont.)

A few examples (cont.)

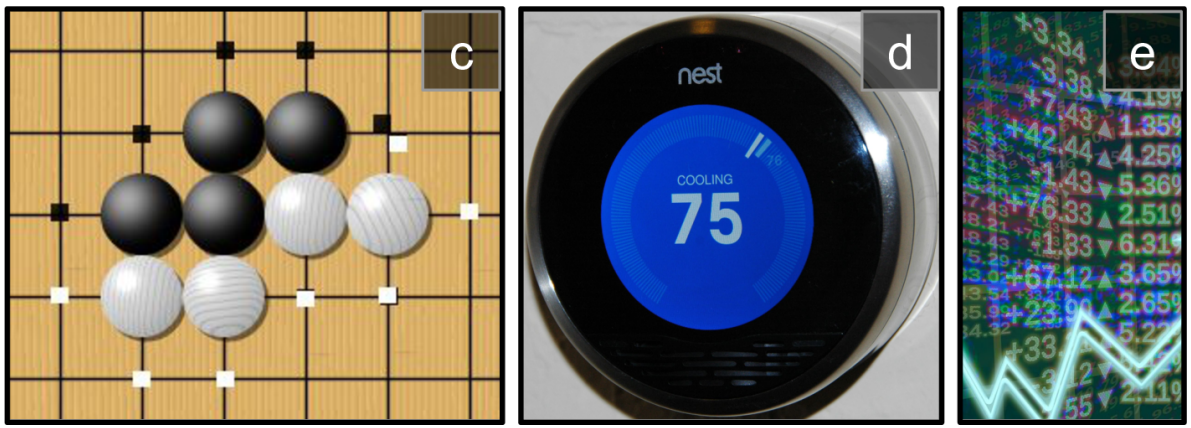


Figure 18-1. Reinforcement Learning examples: (a) robotics, (b) Ms. Pac-Man, (c) Go player, (d) thermostat, (e) automatic trader

(c) The agent does not have to control a physically (or virtually) moving thing. e.g., it can be a smart thermostat, getting rewards whenever it is close to the target temperature and saves energy, and negative rewards when humans need to tweak the temperature, so the agent must learn to anticipate human needs.

Learning to Optimize Rewards (cont.)

A few examples (cont.)

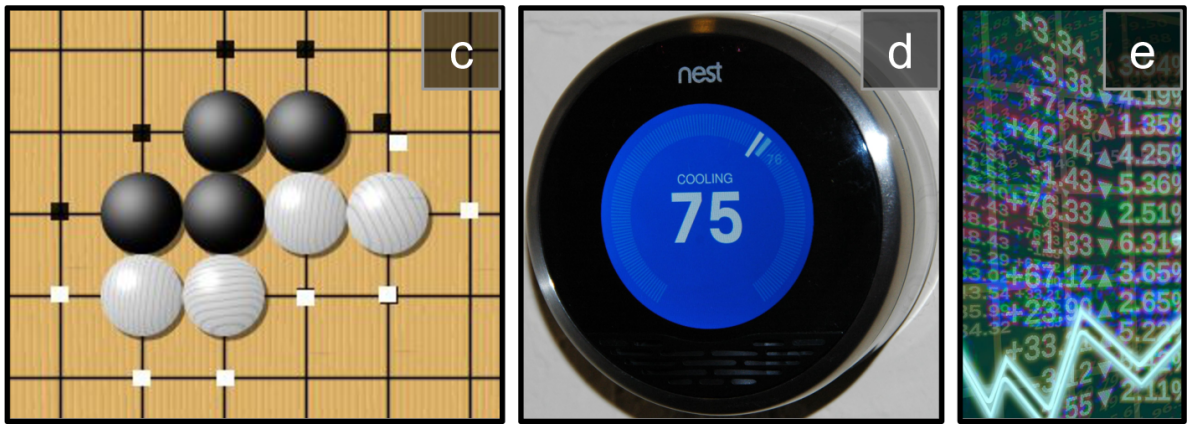


Figure 18-1. Reinforcement Learning examples: (a) robotics, (b) Ms. Pac-Man, (c) Go player, (d) thermostat, (e) automatic trader

(d) The agent can observe stock market prices and decide how much to buy or sell every second. Rewards are obviously the monetary gains and losses.

(e) The agent may be a program to control a robot to move to get out of a maze. In this case, there is only negative reward and no positive reward. So it had better find the exit as soon as possible!

Policy Search

- The algorithm used by the software agent to determine its actions is called its *policy*, e.g., the policy could be a neural network taking observations as inputs and outputting the action to take (see Figure 18-2).

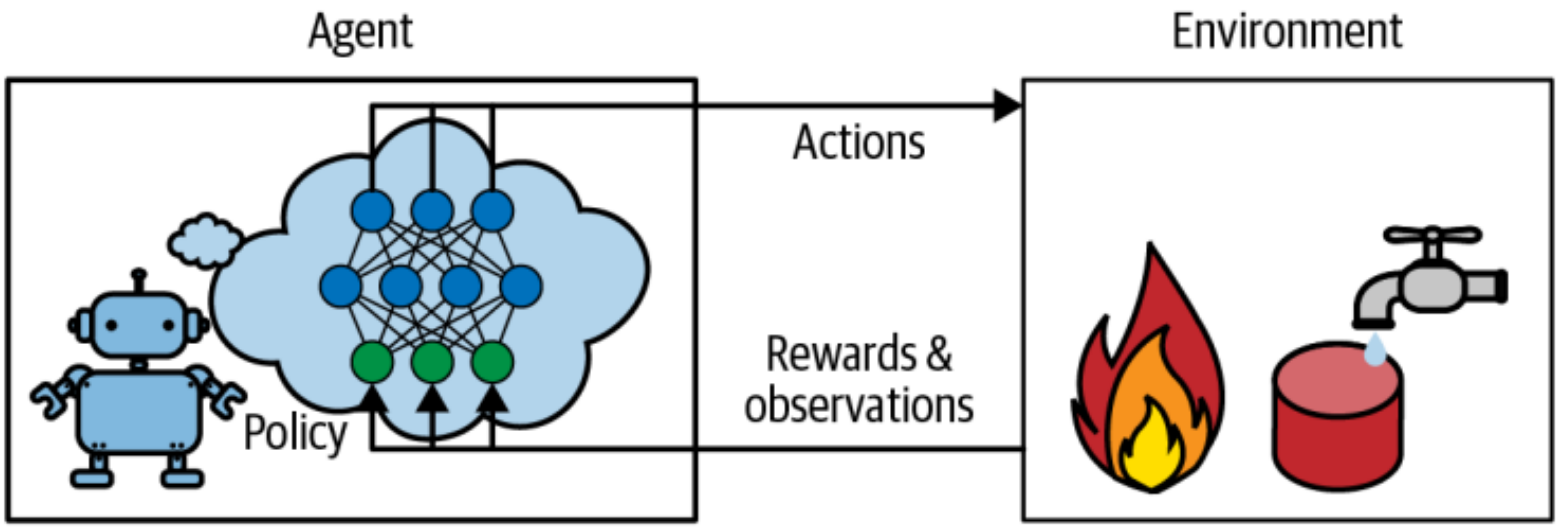


Figure 18-2. Reinforcement Learning using a neural network policy

Policy Search (cont.)

- The policy can be any algorithm you can think of, and it does not even have to be deterministic. For example, a robotic vacuum cleaner whose reward is the amount of dust it picks up in 30 minutes. Its policy could be:
  - move forward with some probability p every second, or
  - randomly rotate left or right (a random angle between $-r$ and $+r$ degrees) with probability $1 - p$.
- There are *policy parameters* that you can tweak. In the vacuum robot above, the policy parameters are p and r; because randomness is involved, it is called a *stochastic policy*. The robot will have an erratic trajectory, which guarantees that it will eventually get to any place it can reach and pick up all the dust.

Policy Search (cont.)

Possible learning algorithms

- One possible learning algorithm would be to find the optimal *policy parameters* by trying all the combinations of parameter values in the *policy space*. This is known as *policy search* and this is a brute force approach.
  When the policy space is large (which is generally the case), finding a good set of parameters this way is like searching for a needle in a gigantic haystack.

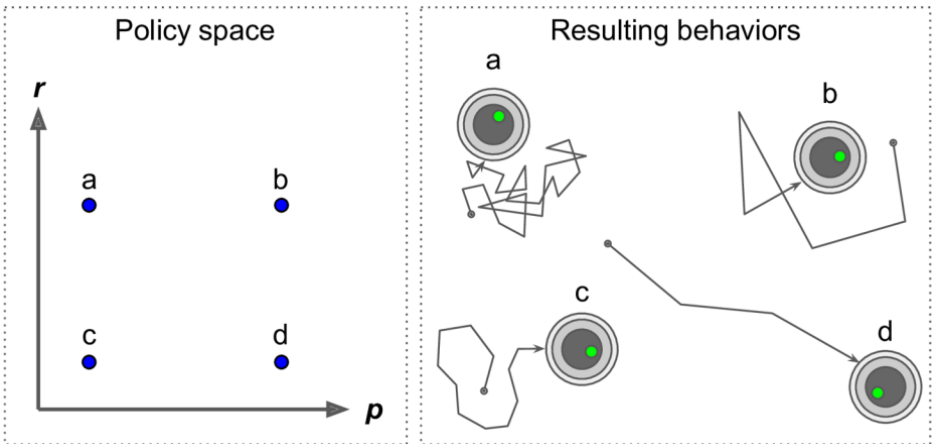


Figure 18-3. Four points in policy space (left) and the agent's corresponding behavior (right)

Policy Search (cont.)

Possible learning algorithms (cont.)

- Another way to explore the policy space is to use *genetic algorithms*. For example, you could randomly create a first generation of 100 policies and try them out, then "kill" the 80 worst policies and make the 20 survivors produce 4 offspring each. An offspring is just a copy of its parent.

  The surviving policies plus their offspring together constitute the second generation. You can continue to iterate through generations this way, until you find a good policy.

Policy Search (cont.)

Possible learning algorithms (cont.)

- A third approach is to use optimization techniques, by evaluating the gradients of the rewards with respect to the policy parameters, then tweaking these parameters by following the gradient toward higher rewards (*gradient ascent*). This approach is called *policy gradients* (PG), which we will see more detail later.

  For example, in the robot vacuum cleaner, you could slightly increase p and evaluate whether this increases the amount of dust picked up by the robot in 30 minutes; if it does, then increase p some more, or else reduce p.

Introduction to OpenAI Gym

- To implement a *policy gradient* (PG) algorithm using TensorFlow, we need to create an environment for the agent to live in. e.g., an Atari game environment for an agent to learn to play an Atari game; a *simulated environment* for training a walking robot.
- *OpenAI gym* is a toolkit that provides a wide variety of simulated environments (Atari games, board games, 2D and 3D physical simulations, and so on), so you can train agents, compare them, or develop new RL algorithms.
- OpenAI Gym can be installed on Colab:

```
# Only run these commands on Colab or Kaggle!
%pip install -q -U gymnasium
```

or on your own computer:

```
pip install gymnasium
```

Introduction to OpenAI Gym

- Example: To create the *CartPole* environment using the `make()` function (this is a 2D simulation where the robot can move left or right):

```
>>> import gymnasium as gym
>>> env = gym.make("CartPole-v1", reader_mode="rgb_array")
>>> obs, info = env.reset(seed=42)
>>> obs
array([ 0.0273956 , -0.00611216, 0.03585979, 0.0197368 ], dtype=float32)
>>> info
{}
```

The four numbers in the observation vector (`obs`) are: **(1)** the cart's horizontal position (0.0 = centre), **(2)** its velocity (positive means *right*), **(3)** the angle of the pole (0.0 = vertical), and **(4)** its angular velocity (positive means *clockwise*.

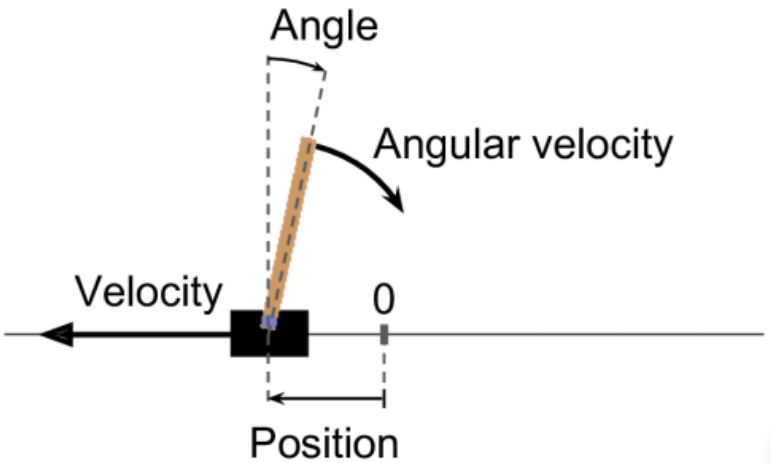
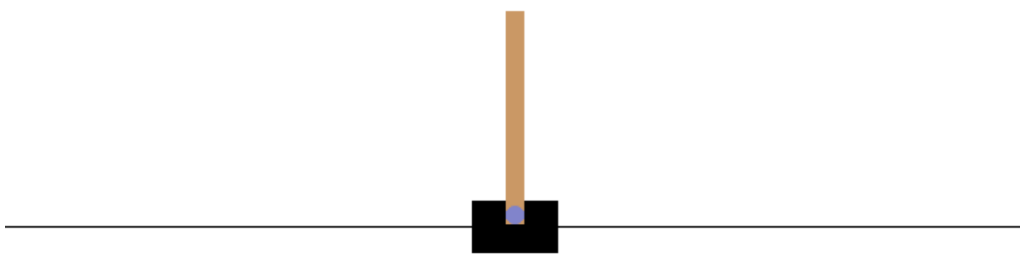


Figure 18-4. The CartPole environment

Rendering the environment

The game environment can be render using the `render()` method and the output image can be displayed using Matplotlib's `imshow()` function:

```
>>> env.reset()
>>> img = env.render()
>>> img.shape # height, width, channels (3 = Red, Green, Blue)
(400, 600, 3)
>>> plt.imshow(img)
```



Useful functions from the OpenAI Gym environment

- `env.action_space` – returns the possible action, e.g., in CartPole example:

```
>>> env.action_space
Discrete(2)
```

where 0 denotes accelerating the robot left and,
1 denotes accelerating the robot right respectively.

Introduction to OpenAI Gym (cont.)

Useful functions from the OpenAI Gym environment

- `env.step(action)` – performs the specified action once.
  e.g., since the pole is leaning toward the right (`obs[2] > 0`), let's accelerate the cart toward the right:

```
>>> action = 1 # accelerate right
>>> obs, reward, done, truncated, info = env.step(action)
>>> obs
array([ 0.02727336, 0.18847767, 0.03625453, -0.26141977], dtype=float32)
>>> reward
1.0
>>> done
False
>>> truncated
False
>>> info
{}
```

A Simple Hard-coded Policy

Let's try a simple policy:
– accelerate left when the pole is leaning toward the left and
– accelerates right when the pole is leaning toward the right.

We will run this policy to see the average rewards it gets over 500 episodes:

```
def basic_policy(obs):
    angle = obs[2]
    return 0 if angle < 0 else 1
```

A Simple Hard-coded Policy (cont.)

```
totals = []
for episode in range(500):
    episode_rewards = 0
    obs, info = env.reset(seed=episode)
    for step in range(200):
        action = basic_policy(obs)
        obs, reward, done, truncated, info = env.step(action)
        episode_rewards += reward
        if done or truncated:
            break

    totals.append(episode_rewards)

>>> np.mean(totals), np.std(totals), np.min(totals), np.max(totals)
(41.698, 8.389445512070509, 24.0, 63.0)
```

Using this policy, the average reward is 41.698; the maximum reward is 63. That is, this policy never managed to keep the pole upright for more than 63 steps.

(See `cits5017-lect09-demo-CartPole1_using-basic-hard-coded-policy.webm` for the visualization.)

Neural Network Policies

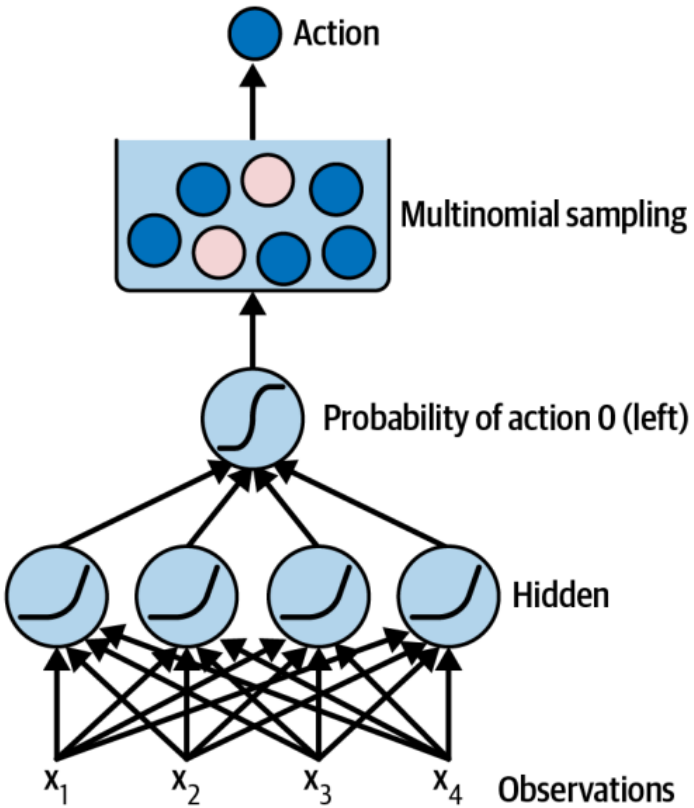


Figure 18-5. Neural network policy

We can train a neural network to create a policy to move the e.g., a network as shown in Figure 18-5:

- The output layer has 1 neuron which outputs the probability p of action 0 (left).
- We will select an action randomly, according to the estimated probabilities. This *exploration/exploitation* dilemma is central in reinforcement learning.
- Note that in this particular environment, the past actions and observations can safely be ignored, since each observation contains the environment's full state.

Neural Network Policies

Things to consider when building and training your network

- If the observations do not contain the full state of the environment, then you may need to consider past actions and observations as well. For example, if the environment only revealed the position of the cart but not its velocity, you would have to consider not only the current observation but also the previous observation in order to estimate the current velocity.
- When the observations are noisy, you generally want to use the past few observations (e.g., by averaging them) to estimate the most likely current state.

Neural Network Policies

Python code for the network in Figure 18-5:

```
import tensorflow as tf

n_inputs = 4        # == env.observation_space.shape[0]

# create the model
tf.random.set_seed(42)
model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(5, activation="relu", input_shape=[n_inputs]),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])

# reset the game environment, for reproducibility
obs, info = env.reset(seed=42)
```

(The Jupyter-Notebook file from the author includes also the animation for the above network with a random policy. See cits5017-lect09-demo-CartPole2_using-NN-random-policy.webm)

But how do we train the network?

Evaluating Actions: The Credit Assignment Problem

- In RL, the only guidance the agent gets is through *rewards*, and rewards are typically sparse and delayed; e.g., if the pole fell after 100 steps, surely the last action at step 100 is not entirely responsible. This is called the *credit assignment problem*: when the agent gets a reward, it is hard for it to know which actions should get credited (or blamed) for it.
- To tackle the above problem, a common strategy is to evaluate an action based on the sum of all the rewards that come after it, usually applying a *discount rate* γ at each step. E.g., if the discount rate $\gamma = 0.8$ and the reward for the next 3 steps are: 10, 0, and -50, then the total score at the current step is: $10 + \gamma \times 0 + \gamma^2 \times (-50) = -22$.

Evaluating Actions: The Credit Assignment Problem

Discounted rewards

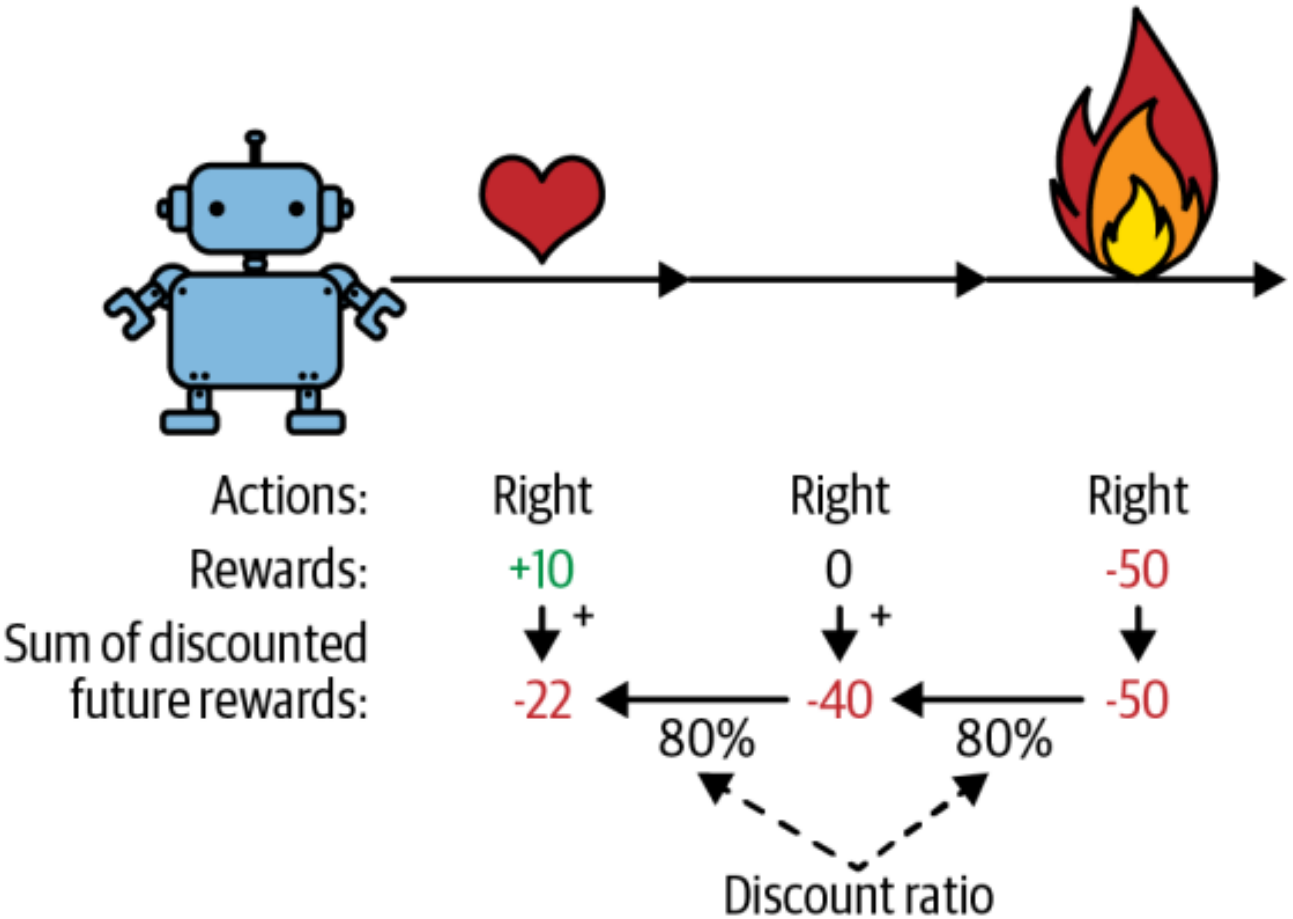


Figure 18-6. Computing an action's return: the sum of discounted future rewards

Evaluating Actions: The Credit Assignment Problem

Discounted rewards (cont.)

- If the discount rate γ is close to 0, then future rewards won't count for much compared to immediate rewards.
- Conversely, if the discount rate γ is close to 1, then rewards far into the future will count almost as much as immediate rewards.
- Typical discount rates are 0.95 or 0.99. With $\gamma = 0.95$, rewards 13 steps into the future count roughly for half as much as immediate rewards (since $0.95^{13} \approx 0.5$), while with $\gamma = 0.99$, rewards 69 steps into the future count for half as much as immediate rewards.
- In the CartPole environment, actions have fairly short-term effects, so choosing a discount rate of 0.95 seems reasonable.

Evaluating Actions: The Credit Assignment Problem

Action advantage

Of course, a good action may be followed by several bad actions that cause the pole to fall quickly, resulting in the good action getting a low score. However, if we play the game many times then, on average, good actions will get a better score than bad ones.

So, to get fairly reliable action scores, we must run many episodes and then normalize all the action scores (by subtracting the mean and dividing by the standard deviation). This calculation gives us a *normalized score* for the action taken at each time step, also referred to as the *action advantage*.

After that, we can reasonably assume that actions with a negative advantage were bad while actions with a positive advantage were good.

Policy Gradients

- *Policy gradients* (PG) algorithms optimize the parameters of a policy by following the gradients toward higher rewards.
- One popular class of PG algorithms, called *REINFORCE algorithms*, was introduced back in 1992[1] by Ronald Williams.
- A common variant of the REINFORCE algorithms is:
  1. Let the neural network policy play the game several times and at each step compute the gradients that would make the chosen action even more likely, but don't apply these gradients yet.
  2. Once you have run several episodes, compute each action's score (i.e., the *action advantage* described in the previous slide).

[1] "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning," R. Williams (1992).

Policy Gradients (cont.)

- A common variant of the REINFORCE algorithms is:
  3. If an action's score is positive, it means that the action was good and you want to apply the gradients computed earlier to make the action even more likely to be chosen in the future. However, if the score is negative, it means the action was bad and you want to apply the opposite gradients to make this action slightly less likely in the future. The solution is simply to multiply each gradient vector by the corresponding action's score.
  4. Finally, compute the mean of all the resulting gradient vectors, and use it to perform a gradient descent step.

Policy Gradients (cont.)

Implementation using tf.keras

We will train the neural network in Figure 18-5 for the CartPole problem.

```
def play_one_step(env, obs, model, loss_fn):
    with tf.GradientTape() as tape:
        left_proba = model(obs[np.newaxis])
        action = (tf.random.uniform([1, 1]) > left_proba)
        y_target = tf.constant([[1.]]) - tf.cast(action, tf.float32)
        loss = tf.reduce_mean(loss_fn(y_target, left_proba))
    grads = tape.gradient(loss, model.trainable_variables)
    obs, reward, done, truncated, info = env.step(int(action))
    return obs, reward, done, truncated, grads
```

Since we are acting as though the chosen action is the best possible action, the target probability must be 1.0 if the chosen action is action 0 (left) and 0.0 if it is action 1 (right). Thus, we have the line:

```
y_target = tf.constant([[1.]]) - tf.cast(action, tf.float32)
```

Policy Gradients (cont.)

Implementation using tf.keras (cont.)

Play enough episodes and collect the rewards and gradients...

```
def play_multiple_episodes(env, n_episodes, n_max_steps, model, loss_fn):
    all_rewards = []
    all_grads = []
    for episode in range(n_episodes):
        current_rewards = []
        current_grads = []
        obs, info = env.reset()
        for step in range(n_max_steps):
            obs, reward, done, truncated, grads = play_one_step(
                env, obs, model, loss_fn)
            current_rewards.append(reward)
            current_grads.append(grads)
            if done or truncated:
                break

        all_rewards.append(current_rewards)
        all_grads.append(current_grads)

    return all_rewards, all_grads
```

Policy Gradients (cont.)

Implementation using tf.keras (cont.)

Define functions for the rewards...

```
def discount_rewards(rewards, discount_factor):
    discounted = np.array(rewards)
    for step in range(len(rewards) - 2, -1, -1):
        discounted[step] += discounted[step + 1] * discount_factor
    return discounted

def discount_and_normalize_rewards(all_rewards, discount_factor):
    all_discounted_rewards = [discount_rewards(rewards, discount_factor)
                              for rewards in all_rewards]
    flat_rewards = np.concatenate(all_discounted_rewards)
    reward_mean = flat_rewards.mean()
    reward_std = flat_rewards.std()
    return [(discounted_rewards - reward_mean) / reward_std
            for discounted_rewards in all_discounted_rewards]
```

We can check that the functions work using some fictitious `rewards` and `discount_rate` parameter values.

Policy Gradients (cont.)

Implementation using tf.keras (cont.)

We are almost ready to train our algorithm. Just need to define some hyperparameters:

```
n_iterations = 150
n_episodes_per_update = 10
n_max_steps = 200
discount_factor = 0.95

optimizer = tf.keras.optimizers.Nadam(learning_rate=0.01)
loss_fn = tf.keras.losses.binary_crossentropy
```

Policy Gradients (cont.)

Implementation using tf.keras (cont.)

Now define our training loop:

```
for iteration in range(n_iterations):
    all_rewards, all_grads = play_multiple_episodes(
        env, n_episodes_per_update, n_max_steps, model, loss_fn)

    # extra code { displays some debug info during training
    total_rewards = sum(map(sum, all_rewards))
    print(f"\rIteration: {iteration + 1}/{n_iterations},"
        f" mean rewards: {total_rewards / n_episodes_per_update:.1f}", end="")

    all_final_rewards = discount_and_normalize_rewards(all_rewards,
                                                       discount_factor)
    all_mean_grads = []
    for var_index in range(len(model.trainable_variables)):
        mean_grads = tf.reduce_mean(
            [final_reward * all_grads[episode_index][step][var_index]
             for episode_index, final_rewards in enumerate(all_final_rewards)
                 for step, final_reward in enumerate(final_rewards)], axis=0)
        all_mean_grads.append(mean_grads)

    optimizer.apply_gradients(zip(all_mean_grads, model.trainable_variables))
```

The `print` statement shows:

```
Iteration: 150/150, mean rewards: 190.3
```

Implementation using tf.keras (cont.)

This is a very impressive performance. The algorithm on average is able to balance the pole for 190.3 time steps!!!

The last thing we want to do is to view the animation:

```
np.random.seed(42)
show_one_episode(pg_policy)
```

where `show_one_episode` takes in the input policy and plays the CartPole game. At each step, an image is rendered. At the end of the game, the list of images generated is passed to the `matplotlib.anomation.FuncAnimation` function to show the animation.
(see the Jupyter-notebook file for Chapter 18 from the author Géron)

(See `cits5017-lect09-demo-CartPole3_using-NN-policy-gradients.webm` for the visualization generated by the `show_one_episode` function.)

Markov Decision Processes

Whereas PG algorithms try to directly optimize the policy to increase rewards, the algorithms we will look at now are less direct: the agent learns to estimate the expected sum of discounted future rewards
– for each state, or
– for each action in each state,
then uses this knowledge to decide how to act.

To understand these algorithms, we must first introduce *Markov decision processes* (MDPs).

Markov chains

In the early 20th century, the mathematician Andrey Markov studied stochastic processes with no memory, called **Markov chains**. Such a process has a fixed number of states, and it randomly evolves from one state s to another s' at each step with a fixed probability. This process depends only on the pair (s, s'), not on past states (the system has no memory).

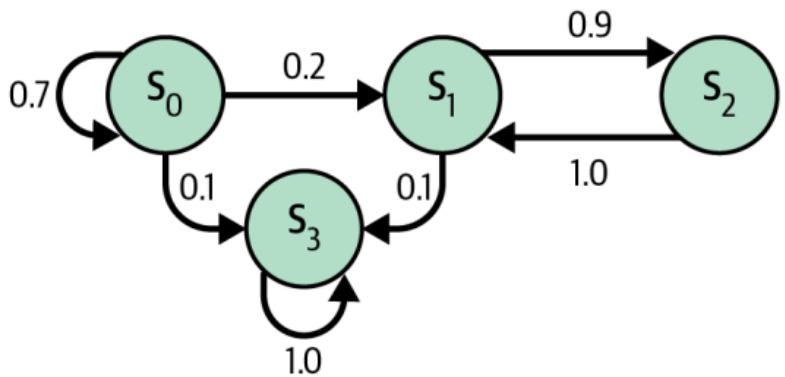


Figure 18-7. Example of a Markov chain

Markov chains

We can use a *state transition matrix* to describe a Markov chain. E.g., the state transition matrix for Figure 18-7 is:

$$\begin{bmatrix} 0.7 & 0.2 & 0.0 & 0.1 \\ 0.0 & 0.0 & 0.9 & 0.1 \\ 0.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

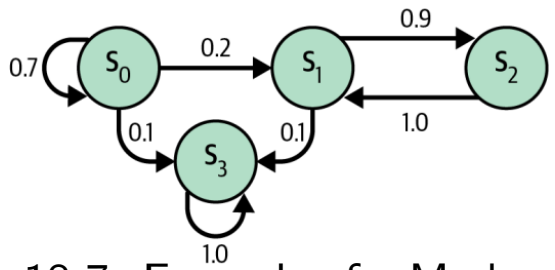


Figure 18-7. Example of a Markov chain

where the row indices (representing the *source states*) and column indices (representing the *destination states*) are in the order of s_0, s_1, s_2, and s_3. E.g., (row 2, column 3) has probability $= 0.9$, denoting the state transition from state s_1 to s_2 has probability 0.9. In a state transition matrix, each row contains probabilities that sum to 1.

Markov Decision Processes

Markov decision processes were first described in the 1950s by Richard Bellman.They resemble Markov chains but with a twist:

- At each step, an agent can choose one of several possible actions, and the transition probabilities depend on the chosen action.
- Moreover, some state transitions return some reward (positive or negative), and the agent's goal is to find a policy that will maximize rewards over time.

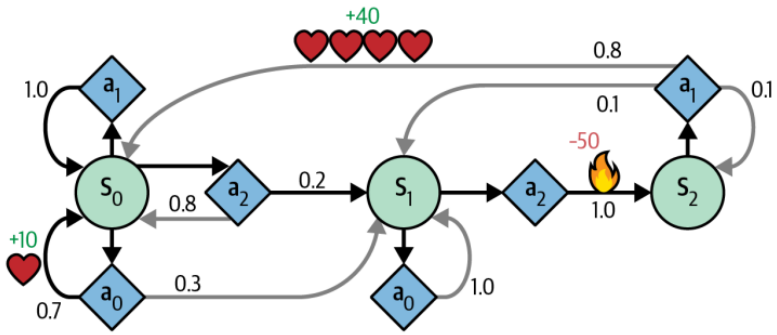


Figure 18-8. Example of a Markov decision process.
Symbols a_0, a_1, and a_2 denote the 3 possible actions.

Markov Decision Processes

By looking at this MDP (Figure 18-8), can we guess which strategy will gain the most reward over time? In state s_0 it is clear that a_0 is the best option (to gain $+10$ reward), but in state s_1 it is not obvious whether the state should stay put (choose action a_0 to remain at s_1) or go through the fire (choose action a_2). For state s_2, the only available action is a_1, which will most likely lead the system back to state s_1 or back to s_0 with a large reward of $+40$.

Bellman proposed a recursive equation (known as the *Bellman Optimality Equation*) which says that

> *the optimal value of the current state is equal to the reward it will get on average after taking one optimal action, plus the expected optimal value of all possible future states that this action can lead to.*

Bellman Optimality Equation

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^*(s') \right] \text{ for all } s$$

- $V^*(s)$ is the *optimal state value* of the state s.
- $T(s, a, s')$ is the transition probability from state s to s' given that the agent chose action a.
- $R(s, a, s')$ is the reward that the agent gets when it goes from state s to s' given that the agent chose action a.
- γ is the *discount rate* (a number in the range $0..1$).

The recursive formulation of the Bellman Optimality Equation can be implemented iteratively using the *value iteration* algorithm. A remarkable result is that, given enough iterations, we are guaranteed to converge to the optimal state values for all the states, corresponding to the optimal policy.

The value iteration algorithm

We can implement the *Bellman Optimality Equation* iteratively using the *value iteration* algorithm below:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right] \text{ for all } s$$

where $V_k(s)$ is the estimated value of state s at the kth iteration of the algorithm.

Note: This algorithm is an example of *dynamic programming*, which breaks down a complex problem (in this case estimating a potentially infinite sum of discounted future rewards) into tractable subproblems that can be tackled iteratively (in this case finding the action that maximizes the average reward plus the discounted next state value).

Q-value iteration algorithm

Knowing the optimal state values is useful for evaluating a policy, but it does not tell the agent explicitly what the optimal action to take at each state. Luckily, Bellman found a very similar algorithm to estimate the optimal *state-action values*, generally called the *Q-Values*.

The optimal Q-Value of the state-action pair (s, a), noted $Q^*(s, a)$, is:

> *the sum of discounted future rewards the agent can expect on average after it reaches the state s and chooses action a, but before it sees the outcome of this action, assuming it acts optimally after that action.*

Q-value iteration algorithm (cont.)

We can start by initializing all the Q-value estimates to zero, then you update them using the *Q-value iteration algorithm* below:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right] \text{ for all } (s,a)$$

Once you have the optimal Q-values, the *optimal policy* (often denoted as $\pi^*(s)$) is easy to define: when the agent is in state s, it should choose the action with the highest Q-value for that state:

$$\pi^*(s) = \underset{a}{\operatorname{argmax}}\, Q^*(s,a).$$

Q-value iteration algorithm (cont.)

Let's apply the Q-Value Iteration algorithm to the MDP in Figure 18-8. Firstly, we define the MDP:

```
transition_probabilities = [ # shape=[s, a, s']
        [[0.7, 0.3, 0.0], [1.0, 0.0, 0.0], [0.8, 0.2, 0.0]],
        [[0.0, 1.0, 0.0], None, [0.0, 0.0, 1.0]],
        [None, [0.8, 0.1, 0.1], None]]
rewards = [ # shape=[s, a, s']
        [[+10, 0, 0], [0, 0, 0], [0, 0, 0]],
        [[0, 0, 0], [0, 0, 0], [0, 0, -50]],
        [[0, 0, 0], [+40, 0, 0], [0, 0, 0]]]
possible_actions = [[0, 1, 2], [0, 2], [1]]
```

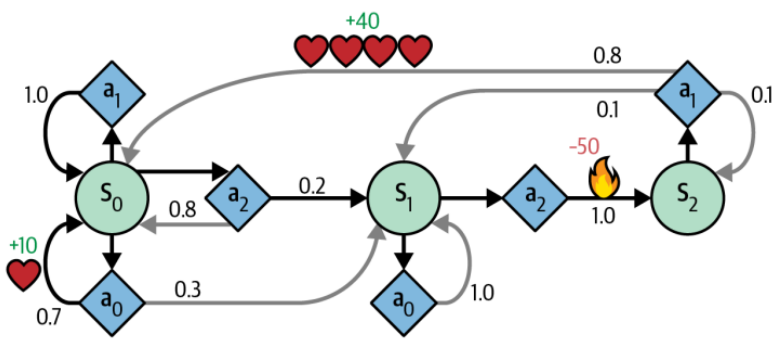


Figure 18-8. Example of a Markov decision process

Q-value iteration algorithm (cont.)

Now let's initialize all the Q-Values to 0 (or to $-\infty$ for those impossible actions) and then run the Q-Value Iteration algorithm:

```
Q_values = np.full((3, 3), -np.inf)  # -np.inf for impossible actions
for state, actions in enumerate(possible_actions):
    Q_values[state, actions] = 0.0  # for all possible actions

gamma = 0.90  # the discount factor

history1 = []  # extra code for generating a figure later on
for iteration in range(50):
    Q_prev = Q_values.copy()
    history1.append(Q_prev) # extra code
    for s in range(3):
        for a in possible_actions[s]:
            Q_values[s, a] = np.sum([
                    transition_probabilities[s][a][sp]
                    * (rewards[s][a][sp] + gamma * Q_prev[sp].max())
                for sp in range(3)])

history1 = np.array(history1)  # extra code
```

Q-value iteration algorithm (cont.)

The resulting Q-Values look like this:

```
>>> Q_values
array([[18.91891892, 17.02702703, 13.62162162],
       [ 0.        ,        -inf, -4.87971488],
       [       -inf, 50.13365013,        -inf]])
>>> np.argmax(Q_values, axis=1)   # optimal action for each state
array([0, 0, 1])
```

The results above say that if the agent was at state s_0, then action a_0 is the optimal action; if at state s_1 then take action a_0 (stay put); at state s_2 there is only one possible action (a_1).

If we change the discount factor γ to 0.95 and rerun the code then we get:

```
...
>>> np.argmax(Q_values, axis=1)   # optimal action for each state
array([0, 2, 1])
```

The optimal action at state s_1 becomes a_2 (going into the fire!)

Temporal Difference Learning

Reinforcement learning problems with discrete actions can be modelled as Markov decision processes, but the agent often does not know what the transition probabilities are (i.e., it does not know $T(s, a, s')$) or what the rewards are going to be (i.e., it does not know $R(s, a, s')$). The agent must experience each state and each transition at least once to know the rewards, and it must go through each state multiple times to get a reasonable estimate of the transition probabilities.

Temporal Difference Learning

The *temporal difference* (TD) learning algorithm is very similar to the *Q-value iteration algorithm*, but tweaked to take into account the fact that the agent has only partial knowledge of the MDP. For example, the agent may use a random *exploration policy* to explore the MDP. The TD learning algorithm updates the estimates of the state values based on the transitions and rewards that are actually observed:

$$V_{k+1}(s) \leftarrow (1-\alpha)V_k(s) + \alpha(r + \gamma\, V_k(s'))$$

or equivalently:

$$V_{k+1}(s) \leftarrow V_k(s) + \alpha\, \delta_k(s, r, s')$$
$$\text{with } \delta_k(s, r, s') = r + \gamma\, V_k(s') - V_k(s)$$

where α is the learning rate; $r + \gamma\, V_k(s')$ is called the *TD target*; $\delta_k(s, r, s')$ is called the *TD error*.

Temporal Difference Learning (cont.)

TD learning has many similarities with stochastic gradient descent, including the fact that it handles one sample at a time. Moreover, just like SGD, it can only truly converge if you gradually reduce the learning rate; otherwise, it will keep bouncing around the optimum Q-values.

For each state s, this algorithm keeps track of a running average of the immediate rewards the agent gets upon leaving that state, plus the rewards it expects to get later, assuming it acts optimally.

Q-Learning

The *Q-learning algorithm* is an adaptation of the *Q-value iteration algorithm* to the situation where the transition probabilities and the rewards are initially unknown. It works by watching an agent play and gradually improving its estimates of the Q-values. Once it has Q-value estimates close enough, the optimal policy is to choose the action that has the highest Q-value (i.e., the greedy policy):

$$Q_{k+1}(s,a) \leftarrow (1-\alpha)Q_k(s,a) + \alpha\left(r + \gamma \max_{a'} Q_k(s',a')\right)$$

where α is the learning rate (e.g., 0.01).

For each state-action pair (s,a), this algorithm keeps track of a running average of the rewards r the agent gets upon leaving the state s with action a, plus the rewards it expects to get later.

Since the target policy would act optimally, we take the maximum of the Q-value estimates for the next state.

Q-Learning

The Q-value at (s_0, a_0) with the discount rate $\gamma = 0.9$:

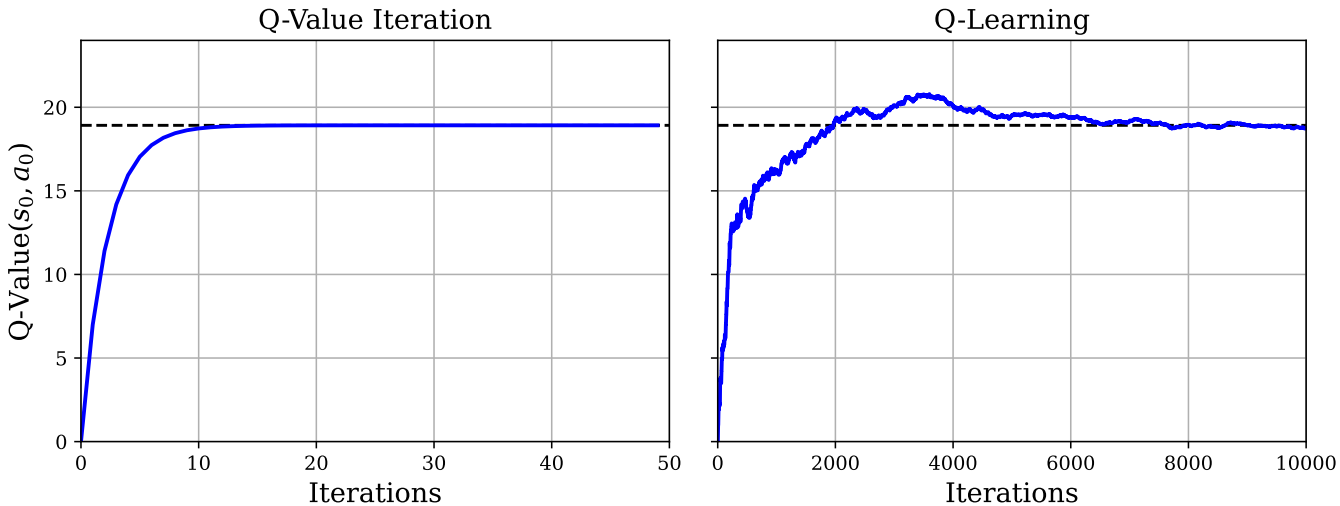
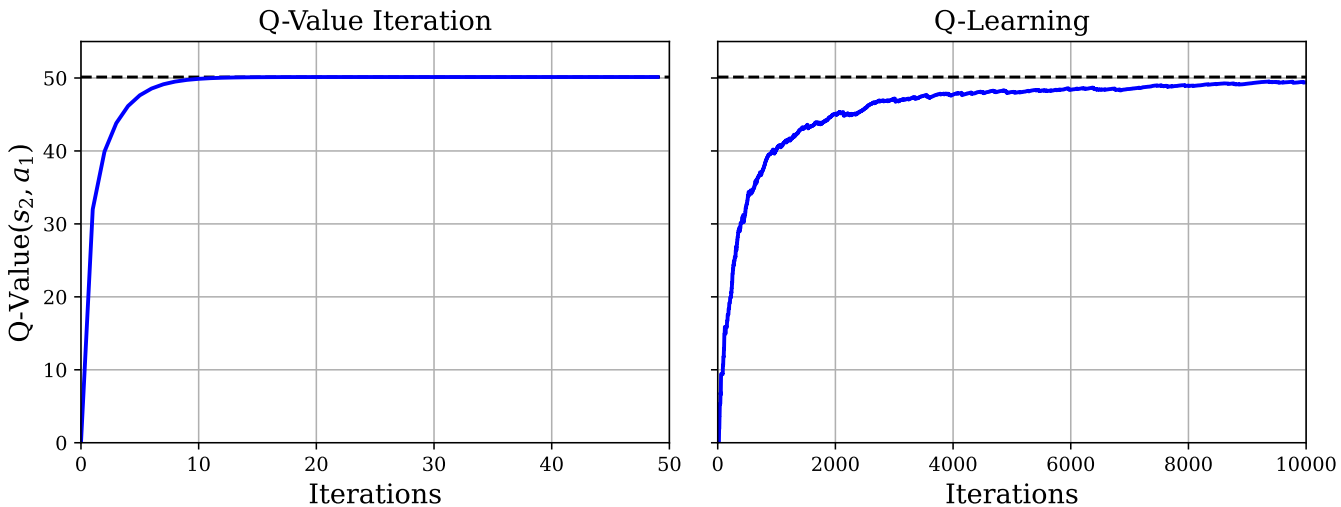


Figure 18-9. Learning curve of the Q-value iteration algorithm versus the Q-learning algorithm

The Q-Value Iteration algorithm (left) converges very quickly, in fewer than 20 iterations, while the Q-Learning algorithm (right) takes about $8,000$ iterations to converge. Obviously, not knowing the transition probabilities or the rewards makes finding the optimal policy significantly harder!

Q-Learning

Similarly, if we compare the Q-values at (s_2, a_1) with the discount rate $\gamma = 0.9$ from the two algorithms, we see that the Q-Learning algorithm took much longer to converge to the ground truth value.



Learning curve of the Q-value iteration algorithm versus the Q-learning algorithm for (s_2, a_1).

Q-Learning

Off-policy and on-policy

The Q-Learning algorithm that we have just implemented is called an *off-policy* algorithm because the policy being trained is not necessarily the one used during training.

Conversely, the Policy Gradients algorithm is an *on-policy* algorithm: it explores the world using the policy being trained.

It is somewhat surprising that Q-Learning is capable of learning the optimal policy by just watching an agent act randomly (imagine learning to play golf when your teacher is a drunk monkey).

For *on-policy* algorithms, we need good exploration policies since the space of the "world" is usually huge.

Q-Learning

The ϵ-greedy policy

Q-Learning can work only if the *exploration policy* explores the MDP thoroughly. We can design a purely random exploration policy. This would guarantee that the algorithm will visit all the states many times. However, this may take a long time to do.
A better option is to use the *ϵ-greedy policy*. The hyperparameter ϵ is often set to a large value close to 1 initially and gradually reduced to a small value (e.g., 0.05) over time.
Thus, initially the Q-Learning algorithm would explore randomly with a high probability ϵ (and only greedily choose the action with highest Q-value with probability $1 - \epsilon$). As time goes by, when ϵ is small, the algorithm would be more greedy in choosing the action which has the highest Q-values.

Approximate Q-Learning

The main problem with Q-Learning is that it does not scale well to large (or even medium) MDPs with many states and actions, as the number of parameters can be too large to be manageable.

The solution is to find a function $Q_\theta(s, a)$ that approximates the Q-Value of any state-action pair (s, a) using a manageable number of parameters (given by the parameter vector θ). This is called *Approximate Q-Learning*.

Approximate Q-Learning

For years it was recommended to use linear combinations of hand-crafted features extracted from the state (e.g., for the Ms. Pac-Man game, the distance of the closest ghosts, their directions, and so on) to estimate Q-Values, but recent research (esp. from DeepMind) showed that using deep neural networks can work much better, and it does not require any feature engineering.

We can train a DNN to estimate the Q-values. Such a network is called a *deep Q-network* (DQN). Using a DQN for Approximate Q-Learning is called *Deep Q-Learning*.

The main idea is to use a manageable number of parameters (represented by the vector $\boldsymbol{\theta}$).

Approximate Q-Learning

Consider the case where a state-action pair (s, a) is given, and we want the approximate Q-value to be **as close as possible** to the reward that we actually observe after playing the action a in state s plus the discounted future rewards. To estimate these future rewards, we can just execute the DQN on the next state s', for all possible actions a'.

Once we have the future rewards estimated, we select the highest one and discount it by the factor γ. Adding it to the reward r at the current state s for taking action a gives us the approximate Q-value.
The target Q-value (Eq. 18-7) is:

$$y(s, a) = r + \gamma \max_{a'} Q_{\boldsymbol{\theta}}(s', a').$$

With this target Q-value, we can run a training step using any gradient descent algorithm to minimize the squared error or the Huber loss between $Q_{\boldsymbol{\theta}}(\cdot, \cdot)$ and the target Q-value.

And that is the *deep Q-learning algorithm*!

Deep Q-Learning

Implementation using tf.keras

```
input_shape = [4]  # == env.observation_space.shape
n_outputs = 2      # == env.action_space.n
model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation="elu", input_shape=input_shape),
    tf.keras.layers.Dense(32, activation="elu"),
    tf.keras.layers.Dense(n_outputs)
])

def epsilon_greedy_policy(state, epsilon=0):
    if np.random.rand() < epsilon:
        return np.random.randint(n_outputs)  # random action
    else:
        Q_values = model.predict(state[np.newaxis], verbose=0)[0]
        return Q_values.argmax()  # optimal action according to the DQN

# We will also need a replay buffer. It will contain the agent's
# experiences, in the form of tuples: (obs, action, reward, next_obs,
# done). We can use the deque class for that.
from collections import deque
replay_buffer = deque(maxlen=2000)
```

Implementation using tf.keras (cont.)

```
# Let's create a function to sample experiences from the replay buffer.
# It will return a list of 6 NumPy arrays: [obs, actions, rewards,
# next_obs, dones, truncateds].
def sample_experiences(batch_size):
    indices = np.random.randint(len(replay_buffer), size=batch_size)
    batch = [replay_buffer[index] for index in indices]
    return [
        np.array([experience[field_index] for experience in batch])
        for field_index in range(6)
    ]  # [states, actions, rewards, next_states, dones, truncateds]

# A function that will use the DQN to play one step, and record its
# experience in the replay buffer
def play_one_step(env, state, epsilon):
    action = epsilon_greedy_policy(state, epsilon)
    next_state, reward, done, truncated, info = env.step(action)
    replay_buffer.append((state, action, reward, next_state, done, truncated))
    return next_state, reward, done, truncated, info
```

Deep Q-Learning

Implementation using tf.keras (cont.)

```
batch_size = 32
discount_factor = 0.95
optimizer = tf.keras.optimizers.Nadam(learning_rate=1e-2)
loss_fn = tf.keras.losses.mean_squared_error

def training_step(batch_size):
    experiences = sample_experiences(batch_size)
    states, actions, rewards, next_states, dones, truncateds = experiences
    next_Q_values = model.predict(next_states, verbose=0)
    max_next_Q_values = next_Q_values.max(axis=1)
    runs = 1.0 - (dones | truncateds)  # episode is not done or truncated
    target_Q_values = rewards + runs * discount_factor * max_next_Q_values
    target_Q_values = target_Q_values.reshape(-1, 1)
    mask = tf.one_hot(actions, n_outputs)
    with tf.GradientTape() as tape:
        all_Q_values = model(states)
        Q_values = tf.reduce_sum(all_Q_values * mask, axis=1, keepdims=True)
        loss = tf.reduce_mean(loss_fn(target_Q_values, Q_values))

    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

Deep Q-Learning

Implementation using tf.keras (cont.)

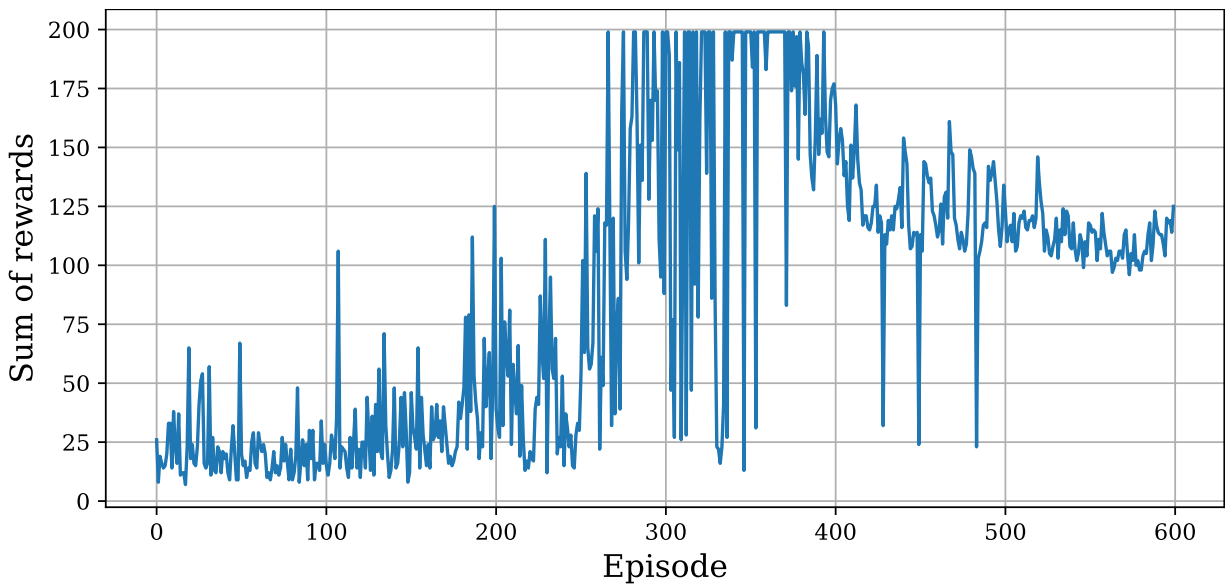
```
# Now, let's train the model!
rewards = []
best_score = 0
for episode in range(600):
    obs, info = env.reset()
    for step in range(200):
        epsilon = max(1 - episode / 500, 0.01)
        obs, reward, done, truncated, info = play_one_step(env, obs, epsilon)
        if done or truncated:
            break
    # extra code - displays debug info
    print(f"\rEpisode: {episode + 1}, Steps: {step + 1}, eps: {epsilon:.3f}",
          end="")
    rewards.append(step)
    if step >= best_score:
        best_weights = model.get_weights()
        best_score = step
    if episode > 50:
        training_step(batch_size)

model.set_weights(best_weights)  # extra code { restores the best model weight
```

Deep Q-Learning

Visualise the rewards that we saved

```
Episode: 600, Steps: 100, eps: 0.010  # my output from the code
                                      # on the previous slide
plt.figure(figsize=(8, 4))
plt.plot(rewards)
plt.xlabel("Episode", fontsize=14)
plt.ylabel("Sum of rewards", fontsize=14)
plt.grid(True)
plt.show()
```



(Note that the result from my run was much worse than that from the textbook.)

Visualise the rewards that we saved (cont.)

As you can see, the algorithm took a while to start learning anything, in part because ϵ was very high at the beginning. It first reached the max reward at around episode 290, but it immediately dropped, then bounced up and down a few times, and soon after it looked like it had finally stabilized near the max reward, at around episode 550, its score again dropped down dramatically. At the last episode, the reward was only 100. This is called *catastrophic forgetting*, and it is one of the big problems facing virtually all RL algorithms.

(See `cits5017-lect09-demo-CartPole4_using-DQN-epsilon-greedy-policy.webm` for the visualization generated by the `show_one_episode` function where the pole tilted over at step 143, which is still not too bad.)

Deep Q-Learning

Conclusion

Reinforcement learning is notoriously difficult, largely because of the training instabilities and the huge sensitivity to the choice of hyperparameter values and random seeds. Many state-of-the-art RL algorithms were trained with millions of steps for long periods of time and the learning rate was set to drop extremely slowly.

(The ending sentence of the chapter from Géron...)
Reinforcement learning is a huge and exciting field, with new ideas and algorithms popping out every day, so I hope this chapter sparked your curiosity: there is a whole world to explore!

Summary

- Understand when to apply *reinforcement learning* (know its applications)
- Understand the concept of *rewards* in RL
- Understand *policy search* and *policy gradients* in RL
- Know how to use the OpenAI Gym toolkit to train a NN for a simple game, such as *CartPole*
- Understand the concept of action evaluation (The Credit Assignment Problem) and discounted rewards
- Understand Markov Decision Processes (MDP)
- Understand Q-learning, ϵ greedy policy, approximate Q-learning, and deep Q-learning
- Know how to implement a deep Q-learning network for a simple problem.

That is all...

This is the last topic of the unit. I hope you enjoyed all the topics as much as I did.

Good luck for the exam!

Have a good week.